Strings & Advanced String Algorithms

KMP · RABIN KARP | INTERMEDIATE TO ADVANCED

A deep dive into the most powerful string matching algorithms in computer science — from the elegant prefix function of Knuth–Morris–Pratt to the rolling hash magic of Rabin Karp. Master the techniques powering text editors, search engines, DNA analysis, and cybersecurity.

Developed by Talenciaglobal

Topic Classification

At a Glance

Category: Advanced Data Structures & Algorithms

Domain: Computer Science → Algorithms → String Matching

Type: Algorithmic techniques with implementation focus

Difficulty: Intermediate to Advanced

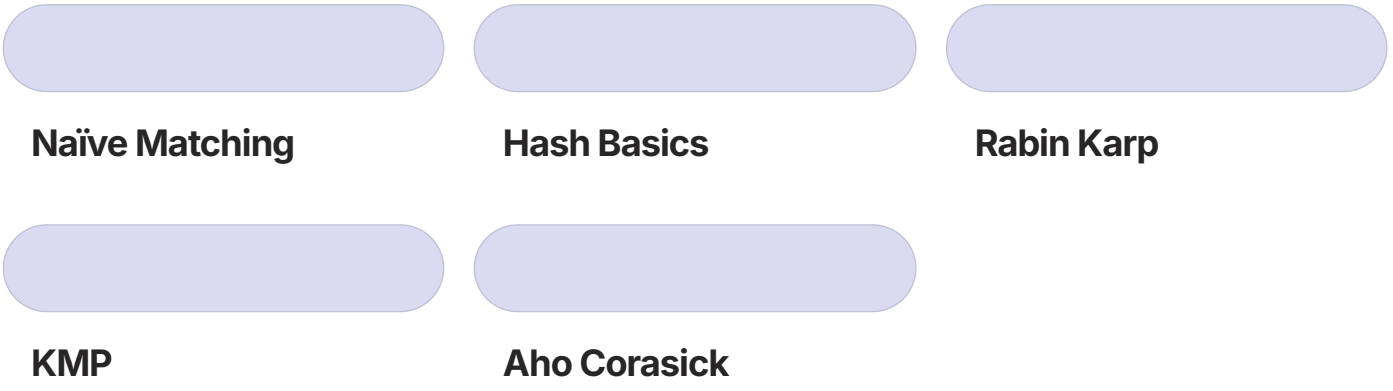
Prerequisites

- Basic string operations
- Hash functions (for Rabin Karp)
- Arrays and prefix concepts (for KMP)
- Big O notation

Industry Relevance

- Text editors (find/replace)
- Search engines (substring search)
- DNA sequence matching
- Plagiarism detection, spam filtering, intrusion detection

Recommended Learning Path



ALGORITHM OVERVIEW

What Is String Matching?

The **string matching problem**: given a text string `T` of length *n* and a pattern string `P` of length *m*, find all starting indices where `T[i..i+m-1] = P`.

1

Naïve Approach

Slide the pattern over the text one position at a time, comparing character by character. Simple but slow — can be `O(n·m)` with many partial matches.

2

KMP Algorithm

Cleverly remembers how much of the pattern matched so far. After a mismatch, shifts the pattern by more than one position using a precomputed **prefix function**. Achieves `O(n+m)` worst case.

3

Rabin Karp

Computes a hash of the pattern, then computes a rolling hash for each text window. Compares only when hashes match. Average `O(n+m)` with O(1) per shift.

Core Algorithms at a Glance

Three fundamental approaches to string matching — each with distinct trade-offs in time, space, and complexity.

Algorithm	Core Data Structure	Time Complexity	Space
Naïve	None	O(n·m)	O(1)
KMP	Prefix array (size m)	O(n+m)	O(m)
Rabin Karp	Hash values (rolling)	Avg O(n+m), worst O(n·m)	O(1)

Key terminology: **pattern, text, shift, match, mismatch, prefix function (π), failure function, LPS** (longest proper prefix which is also suffix), **rolling hash, modulus, base, hash collision, spurious hit**.

Why Do These Algorithms Exist?

The Scale Problem

Searching a 1 GB log file with the naïve method could take **hours**. In DNA alignment, pattern length can be thousands and text length millions — linear time is essential.

Real-Time Demands

Spell check, syntax highlighting, and Ctrl+F require **sub-millisecond** responses. Without efficient algorithms, text editors would freeze on large files.

Security at Scale

Intrusion detection systems must scan network packets for hundreds of attack signatures in real time. Naïve search would be too slow to inspect traffic at gigabit speeds.

Real-World Motivation

- **Ctrl+F in browsers** — highlights all occurrences instantly using variants of Boyer Moore or KMP.
- **Genomics** — finding a specific gene sequence (e.g., SARS-CoV-2) in a genome database requires linear-time algorithms.
- **Spam filters** — scanning emails for known spam patterns using Aho-Corasick (multi-pattern extension of KMP).

Evolution & History

1

1960s

Naïve string search — Baseline O(n·m) algorithm. The starting point for all future work.

2

1970

Morris-Pratt precursor — Early linear time idea, laying groundwork for KMP.

3

1977

KMP published — Knuth, Morris, Pratt publish the first linear worst-case algorithm. Also: Boyer-Moore, often faster in practice.

4

1981

Rabin-Karp — Hashing-based approach; extends to 2D and multiple patterns.

5

1987

Aho-Corasick — Multi-pattern extension of KMP using trie + failure links.

6

1990s+

Hardware acceleration — SIMD, GPU-based search for ultra-fast intrusion detection.

Today, KMP is the canonical linear-time algorithm taught in CS curricula. Rabin Karp is widely used in plagiarism detection (shingling). Many production libraries use hybrid approaches (e.g., two-way algorithm in C's `strstr`).

How KMP Works

Core idea: When a mismatch occurs at position `j` in the pattern, we already know the characters before `j` matched. Use the **prefix function** to determine the next alignment without re-checking matched characters.

Prefix Function (π) Definition

For each position `i` in the pattern, **π[i] = length of the longest proper prefix of P[0..i] which is also a suffix of P[0..i]**.

Example: Pattern "ABABAC"

```
Index:  0  1  2  3  4  5
P:      A  B  A  B  A  C
π:      0  0  1  2  3  0
```

KMP Mismatch Shift — ASCII Diagram

```
Text:    A  B  A  B  A  B  A  C  ...
Pattern: A  B  A  B  A  C
                  ↑ mismatch at position 5 (C vs B)

π[4] = 3 → longest prefix "ABA" equals suffix "ABA"

Shift so prefix "ABA" aligns with suffix "ABA":
Text:    A  B  A  B  A  B  A  C  ...
Pattern:       A  B  A  B  A  C
               ↑ resume comparing at pattern index 3
```

KMP guarantees that each text character is compared **at most twice** — once when advancing, once when falling back — giving the theoretical bound of ≤ 2n comparisons.

KMP Step-by-Step Search

01

Compute Prefix Array

Build the prefix array π for the pattern in O(m) time. This is done once and reused for all searches with the same pattern.

02

Initialize Two Pointers

Set i = 0 (text index) and j = 0 (pattern index). Iterate through the text with i from 0 to n-1.

03

Advance on Match

If T[i] == P[j], increment both i and j. Continue until a mismatch or full match.

04

Handle Full Match

If j == m, record occurrence at index i - m + 1. Then set j = π[j-1] to continue searching for overlapping matches.

05

Handle Mismatch

If j != 0, set j = π[j-1] (do NOT advance i). If j == 0, simply advance i.

How Rabin Karp Works

Core idea: Compare hash values instead of substrings. Use a **rolling hash** to compute the hash of the next window in O(1), making the overall search average O(n+m).

Hash Function

For a window of length *m* with base `b` and modulus `M`:

$$hash(s) = (s[0] \cdot b^{m-1} + s[1] \cdot b^{m-2} + \ldots + s[m-1] \cdot b^{0}) \bmod M$$

Rolling Hash Formula

Given hash `H` for window `T[i..i+m-1]`, compute next window `T[i+1..i+m]`:

$$H_{next} = \left((H - T[i] \cdot b^{m-1}) \cdot b + T[i+m]\right) \bmod M$$

Rolling Hash — ASCII Diagram

```
Text:    A  B  C  D  E
Window 0: A  B  C  →  hash0 = A·b² + B·b + C
Window 1:    B  C  D  →  hash1 = (hash0 - A·b²)·b + D
```

> ⚠ Always verify the actual substring after a hash match! Hash collisions (spurious hits) can cause false positives if you trust the hash alone.

Rabin Karp Step-by-Step

Compute Pattern Hash

Calculate `Hp` — the hash of the entire pattern string. Also precompute `h = b^(m-1) mod M` for use in rolling.

Hash First Window

Compute `Ht` — the hash of the first `m` characters of the text. This is the initial comparison target.

Compare & Verify

For each shift `i` from 0 to n-m: if `Ht == Hp`, verify actual characters to avoid false positives. If match, record index.

Roll the Hash

Compute next window hash using the rolling formula. Handle modulo arithmetic carefully — add `mod` before taking modulo to avoid negative values.

KMP Pseudocode Implementation

Step 1: Compute Prefix Array

```
function computePrefix(pattern):
    m = length(pattern)
    pi = array of size m, initialize pi[0] = 0
    length = 0
    for i from 1 to m-1:
        while length > 0 and pattern[i] != pattern[length]:
            length = pi[length-1]
        if pattern[i] == pattern[length]:
            length = length + 1
        pi[i] = length
    return pi
```

Step 2: KMP Search

```
function KMPsearch(text, pattern):
    n = length(text), m = length(pattern)
    if m == 0: return empty list
    pi = computePrefix(pattern)
    occurrences = empty list
    j = 0  // index for pattern
    for i from 0 to n-1:
        while j > 0 and text[i] != pattern[j]:
            j = pi[j-1]
        if text[i] == pattern[j]:
            j = j + 1
        if j == m:
            occurrences.append(i - m + 1)
            j = pi[j-1]  // continue for overlapping matches
    return occurrences
```

ⓘ The key insight: `j = pi[j-1]` after a match ensures overlapping occurrences are never missed — a common beginner pitfall.

Rabin Karp Pseudocode

```
function rabinKarp(text, pattern, base=256, mod=1000000007):
    n = length(text), m = length(pattern)
    if m == 0 or n < m: return empty list

    // Precompute h = base^(m-1) mod mod
    h = 1
    for i from 1 to m-1:
        h = (h * base) % mod

    // Compute hash of pattern and first window
    patternHash = 0
    textHash = 0
    for i from 0 to m-1:
        patternHash = (patternHash * base + ord(pattern[i])) % mod
        textHash    = (textHash    * base + ord(text[i]))    % mod

    occurrences = []
    for i from 0 to n-m:
        if patternHash == textHash:
            // Verify actual characters (avoid false positives)
            if text[i..i+m-1] == pattern:
                occurrences.append(i)
        // Compute next hash (except for last window)
        if i < n-m:
            textHash = ((textHash - ord(text[i]) * h) % mod + mod) % mod
            textHash = (textHash * base + ord(text[i+m])) % mod

    return occurrences
```

⚠ **Critical:** The `+ mod` before the final `% mod` in the rolling step prevents negative values — a common intermediate-level bug.

Variants & Extensions

KMP Variants

Variant	Modification	Use Case
Standard KMP	Prefix function from pattern	Single pattern search
Extended KMP (Z-function)	Z array on P$T	Simpler implementation, also O(n)
Morris-Pratt	Simpler failure function	Historical
Aho-Corasick	Trie + failure links	Multi-pattern, intrusion detection

Rabin Karp Variants

Variant	Modification	Use Case
Single hash	One modulus	Fast but collision risk
Double hash	Two moduli	Reduce collision probability
Karp-Rabin fingerprint	Mod 2^64 (unsigned overflow)	Very fast, probabilistic
Multi-pattern	Hash set of patterns	Plagiarism detection (shingling)
2D Rabin Karp	Rolling hash in 2D	Image matching

REAL-WORLD USE CASES

Real-World Use Cases

Plagiarism Detection (Rabin Karp)

Industry: Education / Academic Publishing. Break documents into shingles (contiguous substrings of length k). Compute rolling hash for each shingle; compare hash sets. Overlap above threshold suggests plagiarism. Rolling hash allows O(n) extraction of all shingles; hash sets enable fast intersection across millions of document pairs.

Linux grep / IDE Find (KMP Hybrid)

Industry: Software Development. Modern grep uses a combination of algorithms — Boyer-Moore for long patterns, KMP for short. KMP's linear worst case guarantees no catastrophic slowdown even for degenerate patterns like "aaaaa" in "aaaaa...". Consistent sub-second search on large source files.

Intrusion Detection (Aho-Corasick)

Industry: Cybersecurity. Build a trie of all attack signature patterns, add failure links (like KMP), scan text once emitting matches for all patterns simultaneously. Processes gigabit-speed traffic on commodity hardware. Patterns must be stored in memory; large sets require compressed tries.

Hands-On Labs

LAB 1 — BEGINNER

Simulate KMP Prefix Function on Paper

Objective: Manually compute the prefix function to understand the mechanics.

1. Take pattern `"ABABAB"`
2. For each position i=0..5, write the longest proper prefix that is also a suffix
3. Compare with expected π = [0, 0, 1, 2, 3, 4]
4. Repeat with pattern `"AAABAAA"`

Validation: For "ABABAB": i=0:0; i=1:0; i=2:1 (prefix A = suffix A); i=3:2 ("AB"); i=4:3 ("ABA"); i=5:4 ("ABAB").

Learning outcome: The prefix function encodes border lengths — the key insight behind KMP's efficiency.

LAB 2 — ADVANCED

Implement Rabin Karp with Double Hashing

Objective: Implement Rabin Karp with two moduli to eliminate false positives.

1. Use two primes: `MOD1 = 1e9+7`, `MOD2 = 1e9+9`
2. Compute two pattern hashes and two rolling hashes simultaneously
3. Only report match when both hash pairs equal AND string comparison passes
4. Test on adversarial input: `"aaaaaaaa..."` with pattern `"aaa"`

Expected output: No spurious hits; still O(n+m) average.

Learning outcome: Double hashing reduces collision probability to near zero for practical purposes.

Advantages & Disadvantages

KMP — Advantages

- Deterministic O(n+m) — no false positives
- Predictable worst-case performance
- Immune to adversarial inputs
- Streaming friendly — only needs O(m) memory

KMP — Disadvantages

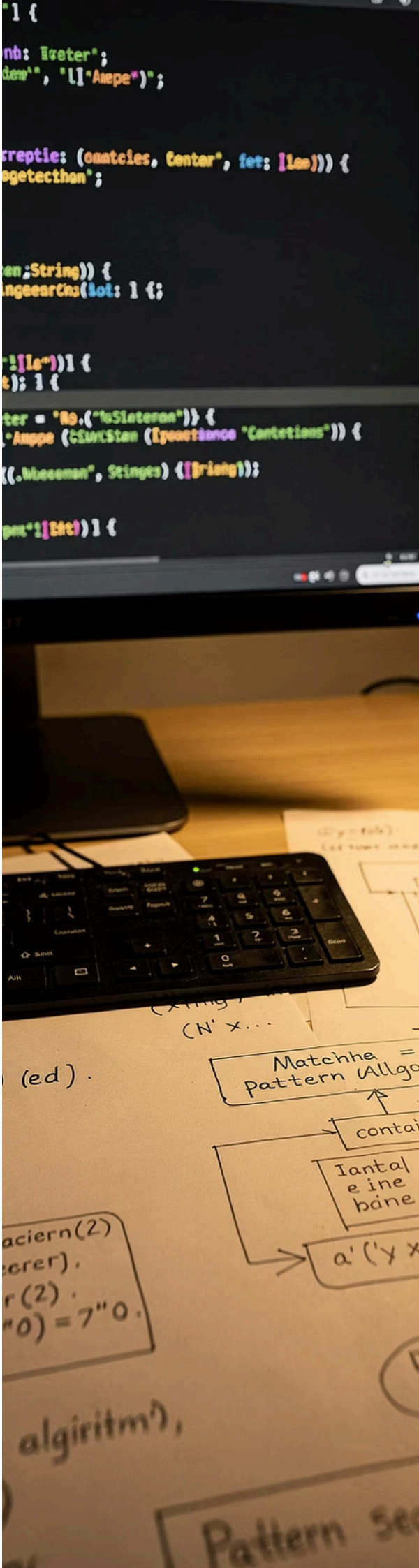
- More complex to implement than naïve
- Extra memory O(m) for prefix array
- Not ideal for very small alphabets with many matches

Rabin Karp — Advantages

- Simple to implement (single hash version)
- Excellent for multiple patterns (hash set)
- Extends to 2D matching
- Large alphabet friendly

Rabin Karp — Disadvantages

- Worst case O(n·m) if many hash collisions
- Modulus selection is critical
- Vulnerable to adversarial inputs with fixed modulus



Performance Considerations

Complexity Analysis

Algorithm	Preprocessing	Search	Space
Naïve	O(1)	O(n·m)	O(1)
KMP	O(m)	O(n)	O(m)
Rabin Karp (single hash)	O(m)	Avg O(n+m), worst O(n·m)	O(1)
Rabin Karp (double hash)	O(m)	Avg O(n+m), lower collision	O(1)

Common Bottlenecks

- KMP: constructing prefix array for long patterns (O(m) — fine in practice)
- Rabin Karp: too-small modulus leads to many collisions and slowdown
- Modular arithmetic with large primes can be slower than direct comparison for short patterns

Optimization Techniques

- KMP: precompute prefix array once; reuse for multiple searches on same pattern
- Rabin Karp: use `base = 256` or larger; use `mod = 2^64` (unsigned overflow in C++) for very fast hashing
- Hybrid: for small m (<20), naïve may be faster due to overhead — switch based on m

Benchmarking guidance: Test on random text (average case), repeated patterns like "AAA...A" (stresses KMP prefix function), and adversarial windows with equal hashes (stresses Rabin Karp).

Scalability & Reliability

Scale	KMP	Rabin Karp
Small ($n < 10^4$)	Overkill — naïve is fine	Overkill
Medium ($n < 10^7$)	Excellent — memory O(m)	Excellent — fast average
Large ($n > 10^9$, e.g., genome)	Single pass, disk streaming possible	Needs rolling hash with disk blocks
Multiple patterns	Use Aho-Corasick (builds on KMP)	Use hash set of patterns

Streaming Compatibility

Both algorithms are streaming-friendly. KMP requires only the pattern and prefix array, scanning text sequentially. Rabin Karp processes window by window without storing the whole text.

Parallelization

Both algorithms are stateless and can be parallelized by splitting text into overlapping chunks. KMP needs careful chunk boundary handling to avoid missing matches that span chunk boundaries.

Genome-Scale Example

Human genome (~3×10^9 bases): KMP scans in linear time with memory ~O(m) — negligible. For 10 TB log analysis, partition across nodes; each node scans its chunk independently with KMP.

Design & Architectural Considerations

Algorithm Selection Guide

Single pattern, worst-case guarantee → KMP

Multiple patterns (fixed set) → Aho-Corasick

Multiple patterns (dynamic) → Rabin Karp with hash set

Long patterns, large alphabet, avg case → Rabin Karp or Boyer-Moore

Embedded systems (tiny memory) → Rabin Karp (O(1) space)

Design Patterns

- **Strategy** — pluggable matching algorithm; switch based on pattern length
- **Template** — generic pattern search with preprocessor and matcher
- **Iterator** — yield matches one by one without storing all results

Maintainability Tips

- Document prefix function logic clearly; use `lps` for longest prefix suffix
- For Rabin Karp, document base and modulus choices explicitly
- Log number of collisions or prefix function computations for debugging

Security Considerations

Algorithmic Complexity Attacks

An attacker can craft input causing Rabin Karp's worst-case O(n·m) if the modulus is predictable (many collisions). **Mitigation:** use a randomized modulus (pick random prime at runtime) or use KMP (immune to this attack).

Timing Attacks

KMP and Rabin Karp can leak information via timing if early exits compare characters. For cryptographic contexts (e.g., password matching), use **constant-time comparison functions** instead.

Hash Flooding

In multi-pattern Rabin Karp, an attacker can generate many strings that hash to the same bucket. **Mitigation:** use double hashing or universal hashing with a randomly chosen hash function.

Best practice for security-sensitive search (e.g., blacklist matching): prefer deterministic algorithms (KMP / Aho-Corasick) to avoid hash collision vulnerabilities entirely.

Best Practices

✅ Do's

Design	Precompute prefix array once per pattern (KMP)
Development	Use built-in string search for very short patterns — library may be faster
Security	Use KMP or Aho-Corasick for adversarial inputs (worst case guaranteed)
Performance	For Rabin Karp, choose base > alphabet size and modulus ~ large prime (10^9+7)
Operations	Monitor pattern length distribution; switch algorithm adaptively

❌ Don'ts

Design	Don't use naïve search for production with unbounded input length
Development	Don't forget to verify substring after hash match in Rabin Karp
Security	Don't use Rabin Karp with fixed modulus in adversarial environments
Performance	Don't compute `h = base^(m-1)` from scratch each window — precompute it
Operations	Don't ignore overlapping matches — KMP needs `j = π[j-1]` after match

Common Mistakes & Pitfalls

1

Off-by-One in Prefix Array (Beginner)

Why it happens: 0-based vs 1-based indexing confusion.
Impact: Wrong shifts, missed or incorrect matches.
Fix: Use consistent indexing throughout; test on small patterns like "AB" and "ABA" first.

2

Forgetting Substring Verification (Beginner)

Why it happens: Trusting hash equality alone in Rabin Karp.
Impact: False positives reported as matches.
Fix: Always do direct character comparison after a hash match.

3

Too-Small Modulus (Intermediate)

Why it happens: Choosing a small modulus for simplicity.
Impact: Many collisions → degrades to O(n·m).
Fix: Use large prime (~10^9+7) or double hash.

4

Negative Modulo After Subtraction (Intermediate)

Why it happens: Language-specific `%` behaviour on negative numbers.
Impact: Wrong hash values, missed matches.
Fix: Adjust: `(hash - char*power + mod) % mod`.

5

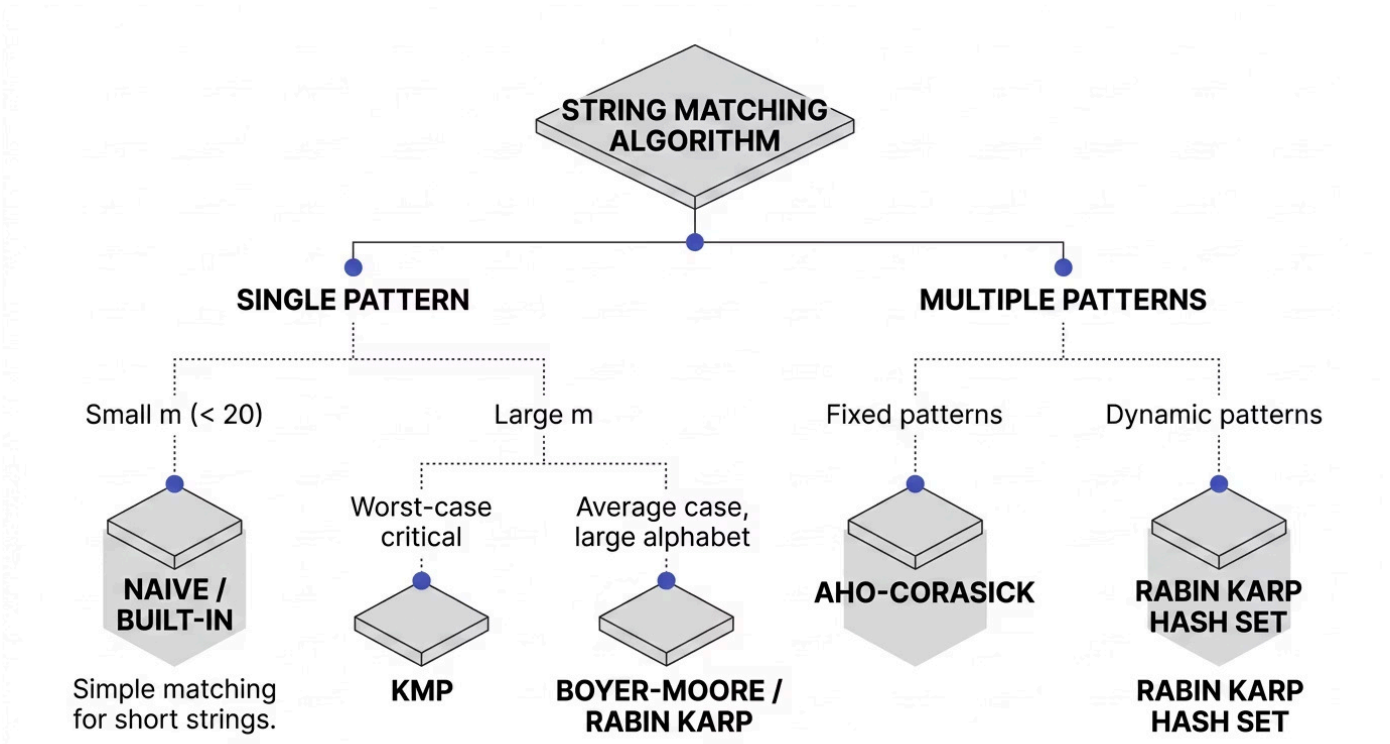
Not Handling Unicode / Multi-byte Characters (Advanced)

Why it happens: ASCII assumption in implementation.
Impact: Incorrect results for non-Latin scripts.
Fix: Use character code points (32-bit) or normalize input before processing.

Comparison with Alternatives

Algorithm	Worst Case	Average	Preprocessing	Space	Best For
Naïve	O(n·m)	O(n·m)	None	O(1)	Very short patterns
KMP	O(n+m)	O(n+m)	O(m)	O(m)	Guaranteed linear time
Rabin Karp	O(n·m)	O(n+m)	O(m)	O(1)	Multiple patterns, large alphabet
Boyer Moore	O(n·m)	O(n/m) sublinear	O(m+σ)	O(m+σ)	Long patterns, large alphabet
Aho-Corasick	O(n + total)	O(n + total)	O(total)	O(total)	Multiple patterns (dictionary)

Decision Tree: Which Algorithm to Use?



Learning Summary & Key Takeaways

1. **KMP achieves O(n+m)** by using a prefix function that tells how much to shift after a mismatch — never re-examining already-matched characters.

2. **Prefix function (π[i])** = length of the longest border (proper prefix = suffix) for each prefix of the pattern. This is the heart of KMP.

3. **Rabin Karp uses rolling hash** to compare pattern and text windows in O(1) per shift. Hash collisions require verification — always compare actual characters.

4. **Rolling hash formula:** `newHash = (oldHash - charOut * base^(m-1)) * base + charIn`

5. **Choose KMP** when worst-case guarantees matter; **choose Rabin Karp** for simplicity and multiple patterns.

6. **Always verify after hash match** in Rabin Karp to avoid false positives (spurious hits).

Revision Cheat Sheet

Concept	Formula / Rule
KMP failure function	π[i] = length of longest proper prefix of P[0..i] that is also a suffix
KMP mismatch transition	j = π[j-1] (if j>0), else i++
Rolling hash (base b, mod M)	H = (H - T[i]·b^(m-1)) · b + T[i+m]
Rabin Karp match condition	patternHash == textHash AND substring equal
Time complexities	KMP: O(n+m); RK avg: O(n+m); RK worst: O(n·m)
Space	KMP: O(m); RK: O(1)

Memory Aids

> "KMP knows its borders" — the prefix function is all about borders (longest prefix = suffix).

> "Hash and roll, but verify your soul" — Rabin Karp's golden rule.

> "When mismatch, don't rewind — use π to find the aligned kind" — KMP shift logic.

Common Interview Questions

01

Compute prefix function for "AABAACAABAA"

Walk through each position applying the KMP prefix function algorithm. Expected: [0,1,0,1,2,0,1,2,3,4,5].

02

Why is KMP linear?

Each text character is compared at most twice — once when advancing pointer `i`, once when falling back via `π`. Total comparisons ≤ 2n.

03

What is a spurious hit in Rabin Karp?

A hash match where the actual strings differ. Caused by hash collisions. Always verify with direct character comparison after a hash match.

04

How does KMP find overlapping matches?

After recording a match, set `j = π[j-1]` and continue scanning — do NOT reset j to 0. This allows the next match to overlap with the previous one.

05

Rabin Karp vs KMP for DNA search (A,C,G,T)?

Small alphabet (4 chars) means many hash collisions for Rabin Karp → more spurious hits → slower. KMP is more predictable and preferred for DNA with small alphabets.

06

How to choose base and modulus for Rabin Karp?

Base > alphabet size (e.g., 256 for ASCII). Modulus = large prime (e.g., 10^9+7). For extra safety, use double hash with two independent primes.

Recommended Next Topics

Aho-Corasick

Multi-pattern extension of KMP using a trie with failure links. Scans text once to find all occurrences of all patterns simultaneously. Essential for intrusion detection and dictionary matching.

Boyer-Moore

Achieves sublinear average case O(n/m) by skipping characters using bad-character and good-suffix heuristics. Often the fastest algorithm in practice for long patterns and large alphabets.

Suffix Array / Suffix Tree

Advanced data structures enabling O(m log n) or O(m) pattern search after O(n log n) or O(n) preprocessing. Ideal when the same text is searched many times with different patterns.

Z Algorithm

Alternative linear-time string matching using a Z array. Computes for each position the length of the longest substring starting there that matches a prefix of the string. Simpler to implement than KMP for some use cases.

Rolling Hash Applications

Beyond string matching: longest common substring, document shingling for plagiarism detection, Rabin fingerprinting in distributed systems, and content-defined chunking for deduplication.